



Implementation of a Data-Driven State-Feedback Controller for a Self-Stabilising Bicycle

Fürchungsarbeit

Vorgelegt an der Universität Stuttgart von
Hadi Elnemr

Master of Science Informationstechnik

Prüfern: Prof. Dr. -Ing. Andrea Iannelli,
Jun.-Prof. Dr.-Ing. Florian Pfaff
Betreuer: M.Sc. Yifan Xie

5. April 2025

Erklärung

Ich erkläre, die Arbeit selbständig verfasst und bei der Erstellung dieser Arbeit die einschlägigen Bestimmungen, insbesondere zum Urheberrechtsschutz fremder Beiträge, eingehalten zu haben. Soweit meine Arbeit fremde Beiträge (z. B. Bilder, Zeichnungen, Textpassagen) enthält, erkläre ich, dass diese Beiträge als solche gekennzeichnet sind (z.B. Zitat, Quellenangabe) und ich eventuell erforderlich gewordene Zustimmung der Urheber zur Nutzung dieser Beiträge in meiner Arbeit eingeholt habe.

Stuttgart, den 10.04.2025

(...)

Contents

List of Figures	VII
List of Tables	IX
Listings	XI
1 Introduction	1
1.1 Bicycle Modelling	1
1.2 Bicycle Control	3
1.3 Data-Driven Control Methods	4
1.4 WHEELTEC MiniBike	5
1.5 Structure of thesis	6
2 Theory	7
2.1 Bicycle Modelling	7
2.2 Data-Driven Min-Max MPC	8
3 Simulations	13
3.1 Linear Quadratic Regulator (LQR)	14
3.2 Data-Driven State-Feedback Controller	15
3.3 Data-Driven Min-Max MPC Controller	17
3.4 Comparisons?	17
4 Experimental Work	18
4.1 Mini-Bike	18
4.2 Experiments	20
5 Conclusion and Future work	23
5.1 Conclusion	23
5.2 Future Work	23
6 Conclusion	24
Bibliography	27

List of Figures

1.1	A picture for the MiniBike used in the experiments.	6
2.1	A Kinetic no-fork model of the bicycle	8
3.1	Lean and steering angles over time under different LQR settings for an initial state vector (in radians) $x_0 = [0.0873, 0, 0]^\top$	15
3.2	Lean and steering angles over time under different data-driven state-feedback settings for an initial state vector (in radians) $x_0 = [0.0873, 0, 0]^\top$, $Q = \text{diag}(300, 0.1, 300)$, $R = 1$, noise bound $\epsilon = 0.001$ and data length of $T = 40$	16
3.3	Lean and steering angles over time under different data-driven MPC settings for an initial state vector (in radians) $x_0 = [0.0873, 0, 0]^\top$, $Q = \text{diag}(300, 0.1, 300)$, $R = 1$, noise bound $\epsilon = 10^{-5}$, data length of $T = 40$, and <i>mpc_iterations</i> = 150.	17
4.1	A picture for the MiniBike used in the experiments.	18
4.2	Lean angle over time under different data-driven state-feedback settings for an initial state vector (in radians) $x_0 = [0.0873, 0, 0]^\top$, $Q = \text{diag}(300, 0.1, 300)$, $R = 1$, noise bound $\epsilon = 0.001$ and data length of $T = 40$	22

List of Tables

3.1	Simulation Parameters	14
4.1	Overview of controllers applied on the mini-bike	20

Listings

Abstract

Riding a bicycle is a common activity for humans, but designing a controller to autonomously stabilise a bicycle presents significant engineering challenges. Traditional model-based controllers, such as linear quadratic regulators (LQR) and model predictive control (MPC), require an accurate system model. However, some parameters of a bicycle, i.e., the centre of gravity, can be difficult to determine and may fluctuate due to varying loads or additional passengers. In this research project, a data-driven state-feedback controller is implemented to stabilise an autonomous mini-bike by solving a semidefinite program (SDP). The method does not identify the system model of the bicycle and directly uses noisy collected data to design the controller. Two models of the bicycle are used to test and assess the controller. Along with the simulations, experiments on a model-scale bicycle demonstrate successful stabilisation using noisy data generated based on the system dynamics.

1 Introduction

Riding a bicycle is a common activity for humans, but designing a controller to autonomously stabilize a bicycle presents significant engineering challenges. Traditional model-based controllers, such as **Linear Quadratic Regulator (LQR)** and **Model Predictive Control (MPC)**, require an accurate system model. However, some parameters of a bicycle, i.e., the center of gravity, can be difficult to determine and may fluctuate due to varying loads or additional passengers. In this project, we aim to design and implement a **Data-Driven Model Predictive Control (DDMPC)** to stabilize an autonomous mini-bike. The method does not identify the system model of the bike and directly uses the collected data to design the controller. The goal is to stabilize the bike at a constant speed and the bike does not fall down. Besides, we aim to keep both the lean angle and steering angle within defined safe limits for safety.

To balance a bicycle, a torque should be generated to prevent falling on its side due to gravity. To achieve this, different techniques for torque generation can be introduced such as adding a device to control the leaning, or by controlling the steering angle of the bicycle while moving. While the bicycle is stationary, a flywheel can generate the required torque to achieve the balancing [1]. Another approach is by using a control moment gyroscope (CMG) which was tested out by He and Zhao [2]. Since controlling the steering angle to produce the balancing torque does not need additional devices, it was considered by numerous researchers and tested against emerging control algorithms. Balancing using the steering is based on steering into the direction of falling. A drawback of using only the steering angle is that the bicycle needs an initial speed before being able to balance.

This research project is an attempt to apply the theoretical DDMPC developed by Xie in [13]. The considered application is a model-scale bicycle at the Institute of **System Theory und Automatic Control (IST)**, University of Stuttgart. The bicycle was chosen as the IST has a rear-steered bicycle which is a more complex system that could be tested against the results achieved from this project. In the following sections, the bicycle modelling and Data-driven MPC are discussed.

1.1 Bicycle Modelling

Before implementing a control strategy for balancing a bicycle, a rigorous system model must be established for most techniques, though some approaches are model-free. System models can be classified as either kinematic or dynamic. The kine-

matic model does not consider the applied forces on the system, while the dynamic model considers the torques and forces interacting with the system. Researchers set different parameters for the dynamic models depending on the bicycle under study. Some parameters could be neglected to simplify the system. Complex systems emerging from accurate modelling demand sophisticated control algorithms, which require more computation power. Examples of dynamic models for the bicycle include the Whipple model [3] and the point mass model [4]. Andersson et al. [5] proposed a robust proportional-integral-derivative (PID) to control a bicycle based on the point mass model.

Should be moved to Control?

This down here is mainly a control paragraph followed by modelling again

Since researchers neglect some parameters in the bicycle models to simplify computations, some uncertainties arise. Therefore, a robust controller can mitigate the errors generated by such models. Robust controllers include the Sliding Mode Control (SMC), which was proposed by Chen and Dao [6], and the LQR, which was proposed by Anjumol and Jisha [7]. The controllers proposed in the literature are largely tested in simulations rather than on a real bicycle. This leads to ignoring the emerging uncertainties from real-life models.

This down here is mainly a repeated intro, modify it and remove what is redundant

The first step towards the design of the controllers that will balance and smooth the motion is to model the bicycle. Bicycles are nonlinear and unstable, and therefore developing mathematical models with high accuracy has been an important area of research in recent times. Bicycle dynamics have been modelled using many techniques, including state-space models, kinetic models, and Linear-Parameter-Varying (LPV) models being important techniques.

One of the primary methods used in bicycle modelling is the LPV approach, which accounts for variations in parameters, such as velocity, that affect the stability of the bicycle. The LPV framework is particularly useful when the bicycle's speed is subject to change, as it enables the controller to adapt to varying conditions.

Cerone et al. (2010) proposed an LPV-based stabilisation method for riderless bicycles, which uses a parameter-varying approach to handle the dynamic changes in speed. The study demonstrated that stability can be maintained between two specific speeds by discretising the continuous-time system, thereby addressing the nonlinearity inherent in the bicycle dynamics [8].

Following paragraphs not convincing

In contrast to LPV models, kinetic models focus on the fundamental forces and motions of the bicycle, including the relation between lean angle, steering angle, and

speed. Tapken et al. (2018) presented a comprehensive study on modeling the kinetic and dynamic aspects of bicycles, employing tools such as CAD modelling and Simscape Multibody to simulate the motion accurately [9].

may need fixes

The kinetic model is essential when dealing with physical prototypes, as it directly links the control inputs to the physical response of the bicycle. The combination of kinetic modeling with state-space representations allows for a more comprehensive analysis, particularly when integrating control algorithms such as LQR or PID controllers.

Persson et al. (2021) conducted a comparative analysis of various control models applied to autonomous bicycles. The study highlighted the advantages of using LPV models for systems with varying speeds, while kinetic models proved effective for low-speed stability. This comparison underlines the importance of model selection based on the specific application requirements [10].

In conclusion, accurate modeling of autonomous bicycles requires a balance between capturing the system's dynamic complexity and maintaining computational efficiency. While LPV models offer flexibility in handling speed variations, kinetic models are indispensable for physical implementations and control integration.

1.2 Bicycle Control

Controlling autonomous bicycles poses significant challenges due to their inherently unstable dynamics and the need to maintain balance while steering. Effective control strategies must account for disturbances, speed variations, and the nonlinear nature of bicycle motion. Over the years, various control methods have been proposed, ranging from classical techniques to more advanced model-based and data-driven approaches.

Classical control techniques such as Proportional-Integral-Derivative (PID) control and Linear Quadratic Regulator (LQR) control have been extensively applied to bicycle stabilization. The PID controller is widely used for velocity and lean angle control due to its simplicity and effectiveness. However, it may not guarantee robust stability, particularly in dynamically changing conditions.

acronyms

The LQR controller, on the other hand, optimises a quadratic cost function, balancing the trade-off between control effort and system stability. Persson et al. demonstrated that LQR provides smoother stabilization compared to PID, especially when the bicycle model is linearized [10]. Despite its advantages, LQR assumes a known and accurate model, limiting its applicability in scenarios with model uncertainties.

To address the limitations of traditional methods, advanced control strategies have

been developed. Andersson et al. (2019) proposed a loop-shaping method to stabilise riderless bicycles. This method focuses on shaping the open-loop transfer function to ensure robust stability across varying speeds [5].

State-feedback control has also gained attention as it directly incorporates the measured states into the control law. Turnwald et al. (2019) demonstrated a successful implementation of state-feedback control in an autonomous bicycle, achieving reliable balance even under external disturbances [11].

A comparative study by Yu et al. (2018) investigated the effectiveness of steering control at low speeds, highlighting that traditional PID control is less effective when the bicycle operates near its tipping point. In contrast, the adaptive state-feedback method provided consistent performance across various speed conditions [12].

Experimental implementations revealed that while traditional methods can stabilise the bicycle at fixed speeds, advanced methods such as loop shaping and state-feedback control provide more flexibility. These methods better handle model uncertainties and external disturbances, which are crucial for autonomous operation.

In summary, bicycle control strategies have evolved from simple PID-based methods to more robust and adaptive approaches. Advanced techniques such as LQR, state-feedback, and loop shaping offer improved performance, particularly when dealing with speed variations and external disturbances.

1.3 Data-Driven Control Methods

The increasing complexity and dynamic nature of autonomous systems, such as self-balancing bicycles, have led to the development of data-driven control methods. These methods bypass the need for precise system modeling by directly utilising measurement data to infer control actions. Data-driven approaches are particularly valuable when model inaccuracies or environmental disturbances are significant.

MPC is a widely used approach for controlling systems with constraints. Traditional MPC relies on accurate mathematical models to predict system behavior. However, DDMPC approaches directly utilize input-output data to predict future states, thus mitigating modeling errors.

One notable approach is the **Data-Driven Min-Max MPC**, which aims to robustly stabilise the system while minimizing the worst-case cost. Xie et al. (2024) proposed a robust min-max MPC framework that incorporates noisy input-state data, formulating the control problem as a semi-definite program (SDP). This method guarantees robust stability and constraint satisfaction while reducing conservatism compared to traditional data-driven MPC approaches [13].

Adaptive data-driven MPC has also been proposed to further improve performance by continuously integrating new data. This adaptive strategy reduces model uncertainty over time, leading to more precise control. The combination of min-max optimization and online adaptation provides enhanced robustness, especially in systems with time-varying parameters [13].

A significant advantage of data-driven MPC is its ability to handle uncertainties without requiring an explicit system model. Compared to classical model-based MPC, data-driven methods are more adaptable when the system's dynamics change. Additionally, the use of online data in adaptive MPC helps reduce conservatism, achieving better closed-loop performance [13].

Yu et al. (2018) demonstrated that data-driven steering control techniques could maintain stability at low speeds, where conventional control strategies often fail. This highlights the practicality of data-driven control in real-world applications [12].

Check if data-driven

Experimental studies have shown that data-driven MPC can effectively stabilise a system, even when the underlying dynamic model is not accurately known. Adaptive schemes have demonstrated improved performance over static data-driven methods by continuously refining the control law based on new data [13].

Data-driven control methods offer a promising approach to managing the inherent uncertainties in autonomous bicycles. By leveraging real-time data, these methods can adapt to changing conditions and maintain robust performance, making them superior to purely model-based techniques in many scenarios.

1.4 WHEELTEC MiniBike

In this research project, a self-balancing mini-bike developed by **WHEELTEC** was utilized as the experimental platform. The goal is to implement a data-driven State-Feedback controller for stabilising the bike. The mini-bike is equipped with various sensors and actuators to enable control and data acquisition. An STM32 Microcontroller is used to control the bike, where the C++ codes used to produce the Hexadecimal file to be burnt on the controller are given with the bicycle.

The bike achieves self-balancing through controlling the steering angle. This is done through a controller computed using LQR combined with a Proportional-Integral-Derivative (PID) controller. The LQR calculates optimal control inputs based on the state feedback to minimize the cost function, while the PID controller fine-tunes the steering speed by adjusting the PWM signals to the servo motor.

While moving with a constant speed, the system's balance is continuously monitored through the gyroscope, which provides real-time data on tilt angles and angular

velocities, along with the steering angle. These values are processed by the microcontroller to compute the necessary adjustments to the steering servo to maintain balance.

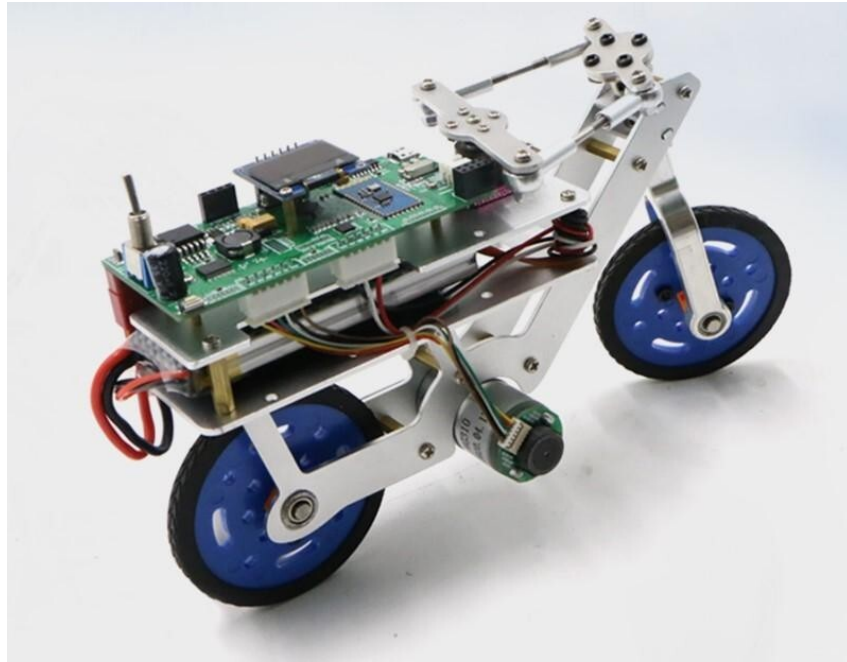


Figure 1.1: A picture for the MiniBike used in the experiments.

1.5 Structure of thesis

This thesis is structured as follows: Chapter 1 provides an introduction, including the background, motivation, literature review, and objectives of the research. Chapter 2 discusses the theoretical foundations, focusing on modelling of a bicycle beside data-driven control techniques. Chapter 3 describes the simulation setup and the implementation of the state-feedback controllers and the simulation results. Chapter 4 presents the experimental work, including the setup and data collection from the mini-bike, along with the results. Chapter 6 concludes the report and discusses future work.

2 Theory

In this section, the theoretical modelling of the bicycle and the data driven Min-Max MPC is discussed.

Since the objective of this research is to simply balance the bicycle, a simple kinetic model is sufficient for this purpose. We use two models and compare between them in terms of control and balance. The difference between both models is considering the fork angle. For controlling the bicycle, the data-driven MPC is compared with an LQR controller and is applied in simulation and on a real model-scale bicycle to assess the controllers. In this chapter, the bicycle model used is discussed and the theory behind the data-driven state-feedback controller is explained.

2.1 Bicycle Modelling

To model the riderless bicycle, it can be treated as an inverted pendulum. There are some (assumptions) parameters that are neglected such as: elasticity of bicycle parts, the tire-road friction, etc. A simple second-order model of the bicycle can be derived given that the rear-speed is constant, the bicycle rolling on the horizontal plane, and initially, the steer axis λ is in a vertical position as in Figure 2.1. Using the Conservation of Angular Momentum, the following Equation 2.1 is acquired to balance the bicycle.

$$\ddot{\varphi} - \frac{g}{h}\varphi = -\frac{av}{hl}\dot{\delta} - \frac{v^2}{hl}\delta, \quad (2.1)$$

where φ is the lean angle, δ is the steer angle, g is the gravity acceleration, h is the height of the center of mass, l is the wheelbase, a is the distance from the center of mass to the front wheel and v is the speed of the bicycle. The bicycle model is a second-order system with two states: lean angle φ and steer angle δ .

This is based on linearisation where the desired stability steady-state for both angles δ and φ is 0.

Now, to consider the fork angle λ , Equation (2.1) is represented as follows.

$$\ddot{\varphi} - \frac{g}{h}\varphi = -\frac{av \sin \lambda}{hl}\dot{\delta} - \frac{(v^2h - ar_\tau g) \sin \lambda}{h^2l}\delta, \quad (2.2)$$

where λ is the fork angle, and r_τ is the trail distance such that $r_\tau = \tan(\frac{\pi}{2} - \lambda)$. It is clear that the difference between both (2.1) and (2.2) is putting the fork angle $\lambda = 90^\circ$ and $r_{tau} = 0$. The state-space representation of the system model considering the fork

angle given by Equation 2.2 is given by

$$\begin{bmatrix} \dot{\varphi} \\ \ddot{\varphi} \\ \dot{\delta} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ \frac{g}{h} & 0 & \frac{(ar_{\tau}g - v^2h) \sin \lambda}{h^2 l} \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \varphi \\ \dot{\varphi} \\ \delta \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{-av \sin \lambda}{hl} \\ 1 \end{bmatrix} \dot{\delta}. \quad (2.3)$$

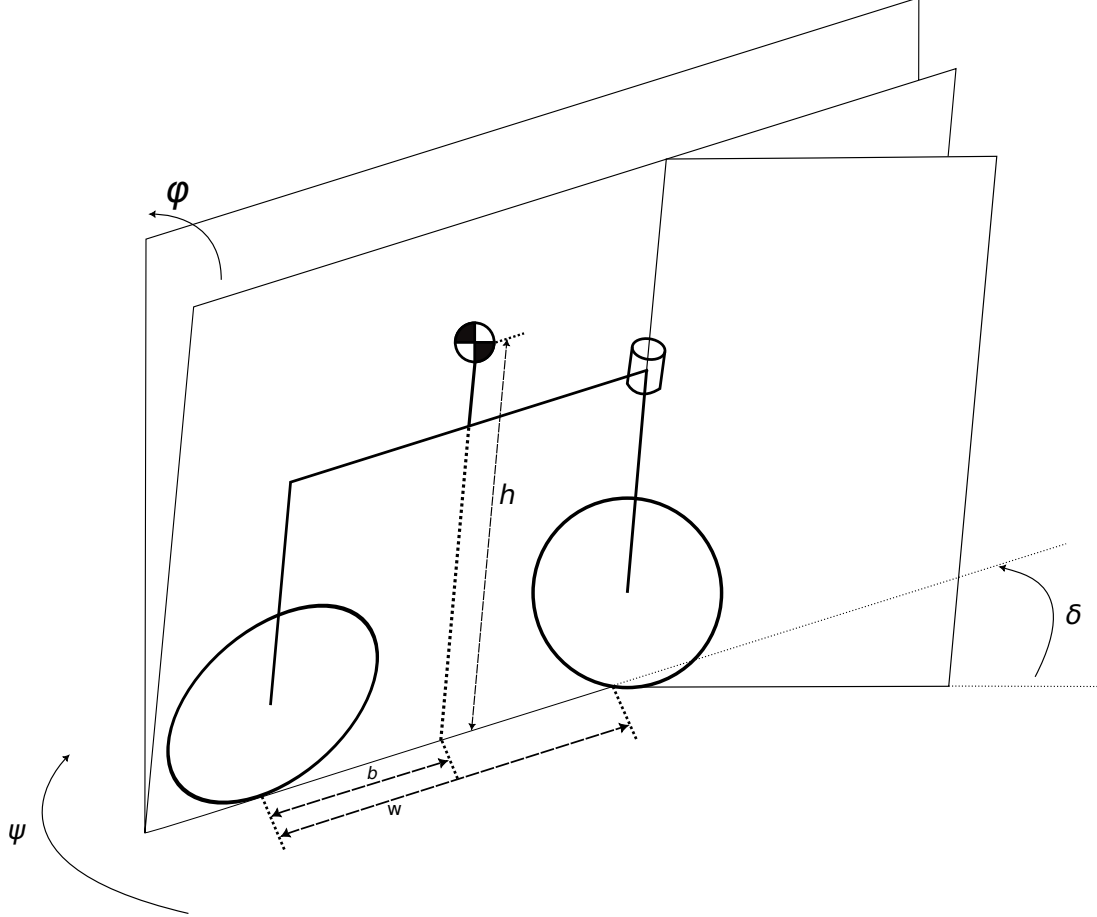


Figure 2.1: A Kinetic no-fork model of the bicycle

Add the Fork Model simple drawing

2.2 Data-Driven Min-Max MPC

In this section, we show how to derive the Data-Driven Min-Max MPC mentioned in [13]. To formulate our problem, we consider a discrete-time linear time-invariant (LTI) system affected by process noise:

$$x_{t+1} = A_s x_t + B_s u_t + \omega_t, \quad (2.4)$$

where $x_t, x_{t+1} \in \mathbb{R}^n$ are the state vectors, $u_t \in \mathbb{R}^m$ is the control input, and $\omega_t \in \mathbb{R}^n$ represents unknown but bounded disturbance or noise at time t . The system matrices

A_s and B_s are assumed to be unknown. Instead of identifying a model explicitly, we utilize available noisy input-state data to design a stabilizing controller. The noise ω_t is assumed to satisfy an upper bound by:

$$\|\omega_t\|_2^2 \leq \varepsilon,$$

where $\varepsilon \geq 0$ is a known bound.

To move to the data-driven framework, let the system equation 2.4 be written in terms of some data sequence satisfying it as follows

$$\omega_i = x_{i+1} - A_s x_i - B_s u_i, \quad (2.5)$$

where $i \in \mathbb{N}$ is the index of the data.

Let (A, B) be the set of system matrices that satisfy the data $x_i, u_i, x_{i+1}; i \in \mathbb{N}$, corresponding to an index in the sequence of data which consists of inputs, noise, and state measurements. This set of system matrices can be written as follows

$$\Sigma_i := \left\{ (A, B) : (2.4) \text{ holds for } \omega_i \text{ satisfying } \|\omega_i\|_2^2 \leq \varepsilon \right\}, \quad (2.6)$$

which can be written as

$$\Sigma_i = \left\{ (A, B) : \begin{bmatrix} I & A & B \end{bmatrix} \begin{bmatrix} I & x_{i+1} \\ 0 & -x_i \\ 0 & -u_i \end{bmatrix} \begin{bmatrix} \varepsilon I & 0 \\ 0 & -I \end{bmatrix} \begin{bmatrix} \star \end{bmatrix}^\top \succeq 0 \right\}, \quad (2.7)$$

such that $\begin{bmatrix} \star \end{bmatrix}^\top$ is used as in the following.

$$ABA^\top = AB \begin{bmatrix} \star \end{bmatrix}^\top$$

Then, the set of system matrices consistent with all data points can be described as follows

$$\mathcal{C} = \bigcap_{i=0}^{T-1} \Sigma_i. \quad (2.8)$$

The set $\mathcal{C}$ can be further represented as

$$\mathcal{C} = \left\{ (A, B) : \begin{bmatrix} I & A & B \end{bmatrix} \Pi(\tau) \begin{bmatrix} I & A & B \end{bmatrix}^\top \succeq 0, \forall \tau = (\tau_0, \dots, \tau_{T-1}), \tau_i \geq 0, i \in \mathbb{I}_{[0, T-1]} \right\}, \quad (2.9)$$

such that Π is

$$\Pi(\tau) = \sum_{i=0}^{T-1} \tau_i \begin{bmatrix} I & x_{i+1} \\ 0 & -x_i \\ 0 & -u_i \end{bmatrix} \begin{bmatrix} \epsilon I & 0 \\ 0 & -I \end{bmatrix} \begin{bmatrix} I & x_{i+1} \\ 0 & -x_i \\ 0 & -u_i \end{bmatrix}^\top. \quad (2.10)$$

mention
quadratic ma-
trix inequality
(QMI)

To design a control law that stabilises the system despite the uncertainty in (A, B) , we setup the problem by defining a stage cost function as

$$l(u, x) = \|u\|_R^2 + \|x\|_Q^2,$$

where $R, Q \succ 0$. Moreover, we define state and input ellipsoidal constraints by the following

$$\|u_t\|_{S_u} \leq 1, \quad \|x_t\|_{S_x} \leq 1, \quad \forall t \in \mathbb{N},$$

where $\|x\|_P = \sqrt{x^\top P x}$. Here, $S_u \succ 0$ and $S_x \succeq 0$. Now, we can state a data-driven min-max MPC problem at time t by

$$J_\infty^*(x_t) := \min_{\bar{u}(t)} \max_{(A, B) \in \mathcal{C}} \sum_{k=0}^{\infty} l(\bar{u}_k(t), \bar{x}_k(t)) \quad (2.11a)$$

$$\text{s.t.} \quad \bar{x}_{k+1}(t) = A\bar{x}_k(t) + B\bar{u}_k(t), \quad (2.11b)$$

$$\bar{x}_0(t) = x_t, \quad (2.11c)$$

$$\|\bar{u}_k(t)\|_{S_u} \leq 1, \quad \forall t \in \mathbb{N}, \quad (2.11d)$$

$$\|\bar{x}_k(t)\|_{S_x} \leq 1, \quad \forall (A, B) \in \mathcal{C}, \quad t \in \mathbb{N}. \quad (2.11e)$$

This cost function can be explained by being a minimisation of the worst-case cost l over all system matrices in $\mathcal{C}$ by adjusting the input $\bar{u}_k(t), \forall k \in \mathbb{N}$ to the system. $\bar{x}_k(t)$ is the predicted state and $\bar{u}_k(t)$ is the control input, both at time step $t + k$. The sets (A, B) used are consistent with the system dynamics in (2.11b). We let the initial state be the measurement state x_t as shown in constraint (2.11c). The ellipsoidal constraints for the control input and states are given in constraints (2.11d) and (2.11e) respectively.

Now, the data-driven min-max MPC can be simplified by just considering a state-feedback control law, such that $u_t = F_t x_t$, where $F_t \in \mathbb{R}^{m \times n}$. It can then be formulated as a Semi-definite Program (SDP) in the following paragraphs.

In [13], it is shown that solving the SDP (2.12) for the min-max MPC (2.11), without the input state constraints, minimises the state-feedback gain F_t that upper bounds the optimal cost.

Suppose that there exist $\gamma > 0$, $H \in \mathbb{R}^{n \times n}$, $L \in \mathbb{R}^{m \times n}$, $\tau \in \mathbb{R}^T$ such that the

inequalities (2.12) hold

$$\begin{bmatrix} 1 & x_t^\top \\ x_t & H \end{bmatrix} \succeq 0, \quad (2.12a)$$

$$\begin{bmatrix} \begin{bmatrix} -H & 0 \\ 0 & 0 \end{bmatrix} + \Pi(\tau) & \begin{bmatrix} 0 \\ H \\ L \end{bmatrix} & 0 \\ \begin{bmatrix} 0 & H & L^\top \\ & 0 & \end{bmatrix} & -H & \Phi^\top \\ & \Phi & -\gamma I \end{bmatrix} \prec 0, \quad (2.12b)$$

$$\tau = (\tau_0, \dots, \tau_{T-1}), \tau_i \geq 0, \forall i \in \mathbb{I}_{[0, T-1]}, \quad (2.12c)$$

where $\Phi = \begin{bmatrix} M_R L \\ M_Q H \end{bmatrix}$ and $M_R^\top M_R = R$, $M_Q^\top M_Q = Q$. The γ here upper bounds the optimal cost in (2.11) without considering the input and state constraints.

To incorporate the remaining constraints, they can be expressed as the following LMIs (2.13) [13].

$$\begin{bmatrix} H & L^\top \\ L & S_u^{-1} \end{bmatrix} \succeq 0, \quad (2.13a)$$

$$\begin{bmatrix} S_x & I \\ I & H \end{bmatrix} \succeq 0. \quad (2.13b)$$

Now, having (2.12) and (2.13), the data-driven min-max MPC (2.11) can be solved after representing it in its SDP formulation as the following.

$$\underset{\gamma, H, L, \tau}{\text{minimise}} \gamma \quad (2.14a)$$

$$\text{s.t. (2.12) and (2.13) hold.} \quad (2.14b)$$

The optimal solution for problem (2.14) is then expressed as γ^* , H^* , L^* , τ^* . Consequently, the optimal state-feedback gain is $F = L^*(H^*)^{-1}$.

Since MPC is a receding horizon control, (2.14) is solved in receding horizon manner as per the Algorithm 1.

Algorithm 1 Data-driven min-max MPC

- 1: At time $t = 0$, measure state x_0 ;
 - 2: Solve the problem (2.14);
 - 3: Apply the input $u_t = F_t x_t$;
 - 4: Set $t = t + 1$ and go back to 2;
-

Main Optimization Problem: Data-Driven Min-Max MPC

Given collected input-state data, the data-driven min-max control problem seeks a state-feedback gain F_t minimizing the worst-case infinite-horizon cost:

$$\min_{\bar{u}(t)} \max_{(A,B) \in \mathcal{C}} \sum_{k=0}^{\infty} x_k^\top Q x_k + \bar{u}_k^\top R u_k$$

given an initial state x_t subject to some input and state constraints and the corresponding consistent set of system matrices $\mathcal{C}$.

The SDP reformulation of the optimisation problem is as follows:

$$\begin{aligned} & \underset{\gamma, H, L, \tau}{\text{minimise}} \gamma \\ & \text{s.t. the following LMIs} \end{aligned}$$

$$\begin{aligned} & \begin{bmatrix} 1 & x_t^\top \\ x_t & H \end{bmatrix} \succeq 0, \\ & \begin{bmatrix} \begin{bmatrix} -H & 0 \\ 0 & 0 \end{bmatrix} + \Pi(\tau) & \begin{bmatrix} 0 \\ H \\ L \end{bmatrix} & 0 \\ \begin{bmatrix} 0 & H & L^\top \\ 0 & 0 & 0 \end{bmatrix} & -H & \Phi^\top \\ & \Phi & -\gamma I \end{bmatrix} \prec 0, \\ & \tau = (\tau_0, \dots, \tau_{T-1}), \tau_i \geq 0, \forall i \in \mathbb{I}_{[0, T-1]}, \end{aligned}$$

$$\begin{aligned} & \begin{bmatrix} H & L^\top \\ L & S_u^{-1} \end{bmatrix} \succcurlyeq 0, \\ & \begin{bmatrix} S_x & I \\ I & H \end{bmatrix} \succcurlyeq 0. \end{aligned}$$

3 Simulations

To assess the stabilisation performance after solving the optimisation problem for the bike, some simulations were conducted. The models of the bicycle from Section 2.1 are tested to understand the mathematical behaviour of the bicycle under different controllers.

As stated before, two models of the bicycle were considered:

- **Without Fork Model:** A simplified kinetic model assuming the fork angle is negligible.
- **With Fork Model:** A kinetic model including the fork angle, accounting for the steering head geometry.

Two versions of the data-driven state-feedback controllers were tested. The first version is a data-driven min-max by solving the SDP in Equation (2.14) only once for an initial state. While the second version is a data-driven min-max MPC where the SDP is solved for each time step. To evaluate the data-driven control laws performance, they are compared to a state-feedback LQR controller.

The simulations were performed using MATLAB based on the kinetic models described in Section 2.1. The following assumptions were made:

- The bicycle moves on a horizontal plane.
- The rear wheel maintains a constant forward velocity.
- Small-angle approximations are applied for the lean angle (ϕ) and steering angle (δ).

The key physical parameters used are summarized in Table 3.1.

Table 3.1: Simulation Parameters

Parameter	Value	Unit	Description
h	0.088	m	Height of center of mass
b	0.055	m	Wheelbase
λ	75 or 90	degrees	Fork angle: angle between the steering axis and the horizontal line
c	0.087	m	Trail distance
r_w	0.0375	m	Radius of the wheel
r_τ	$r_w \cdot \tan\left(\frac{\pi}{2} - \lambda\right)$	m	Distance between front wheel ground contact point and intersection of fork with ground
g	9.81	m/s ²	Gravitational acceleration
v	0.634	m/s	Forward speed
w	0.167	m	Distance between front and rear wheels at ground contact points

The input and state constraints for the system were defined as follows.

$$S_u = 0.005836,$$

$$S_x = \begin{bmatrix} 2.6798 & 0 & 0 \\ 0 & 0.32828 & 0 \\ 0 & 0 & 2.05 \end{bmatrix}$$

The lean angle is constrained between -40 and 40

The three different simulations corresponding to different control strategies are discussed below.

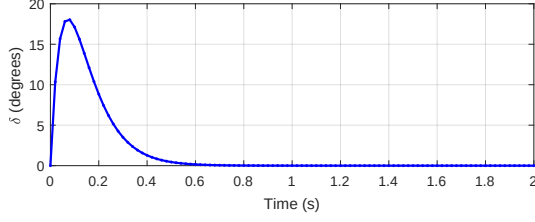
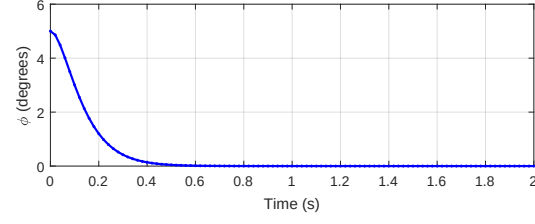
3.1 Linear Quadratic Regulator (LQR)

An LQR controller was designed separately for both the fork and no-fork models by solving the discrete-time Algebraic Riccati Equation. Different weighting matrices Q and R were selected to modify the weights given to the states and inputs respectively, where the weights are used to compute the closed loop cost.

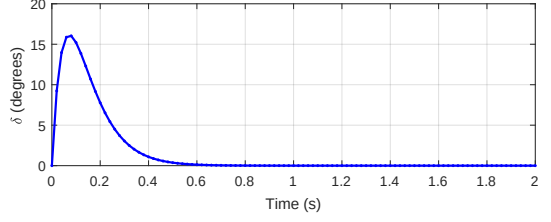
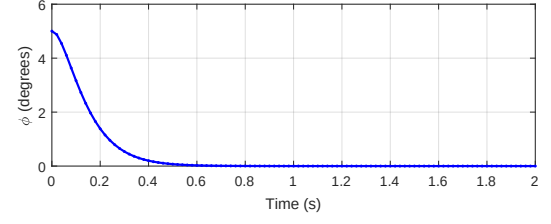
The LQR served as a baseline controller and was also tested to generate input-state data necessary for the subsequent data-driven methods. In Figure 3.1, four simulations are conducted to show the difference in performance for different Q and R settings. Matrix Q corresponds to the cost associated with the states, whereas R corresponds to the cost associated with the inputs. Figure 3.1d shows that giving more weight to the second state, namely the lean angle rate of change, smooths the lean angle response and reduces its steepness. Since the input is the steering angle

rate of change $\dot{\delta}$, when giving it more weight in Figure 3.1c all states are smoothened.

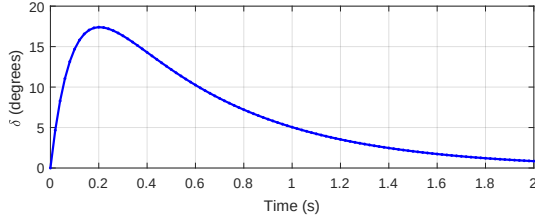
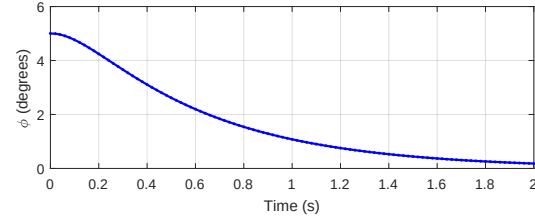
I am here now



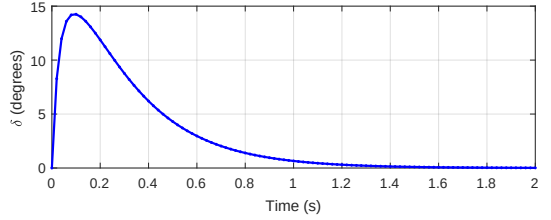
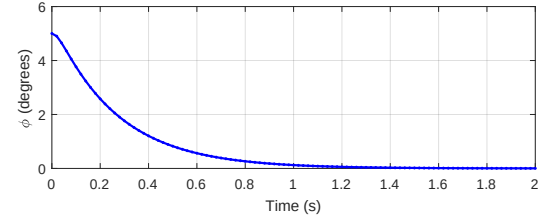
(a) LQR with $Q = \text{diag}(300, 0, 300)$, $R = 1$, 75° fork angle.



(b) LQR with $Q = \text{diag}(300, 0, 300)$, $R = 1$, 90° fork angle.



(c) LQR with $Q = \text{diag}(300, 0, 300)$, $R = 100$, 90° fork angle.



(d) LQR with $Q = \text{diag}(300, 300, 300)$, $R = 1$, 90° fork angle.

Figure 3.1: Lean and steering angles over time under different LQR settings for an initial state vector (in radians) $x_0 = [0.0873, 0, 0]^\top$.

3.2 Data-Driven State-Feedback Controller

Using input-state data, a data-driven state-feedback controller was constructed by solving a one-shot SDP problem mentioned in Equation (2.14). This controller was obtained without knowledge of the exact system model, relying solely on the data and solving the optimisation problem.

This controller depends on obtaining input-state data. Therefore, a range of data was synthesised in simulation given an initial starting point for the system, an error bound for adding noise, and being controlled with a discrete LQR that stabilises the

bicycle. The synthesised data formed a sequence where at each time step it was distorted with additive noise $\omega_i \in \mathbb{R}^n$ with independent components uniformly sampled as

$$\omega_i^j \sim \mathcal{U} \left[-\frac{\epsilon}{\sqrt{n}}, \frac{\epsilon}{\sqrt{n}} \right], \quad j = 1, \dots, n,$$

which can be compactly written as

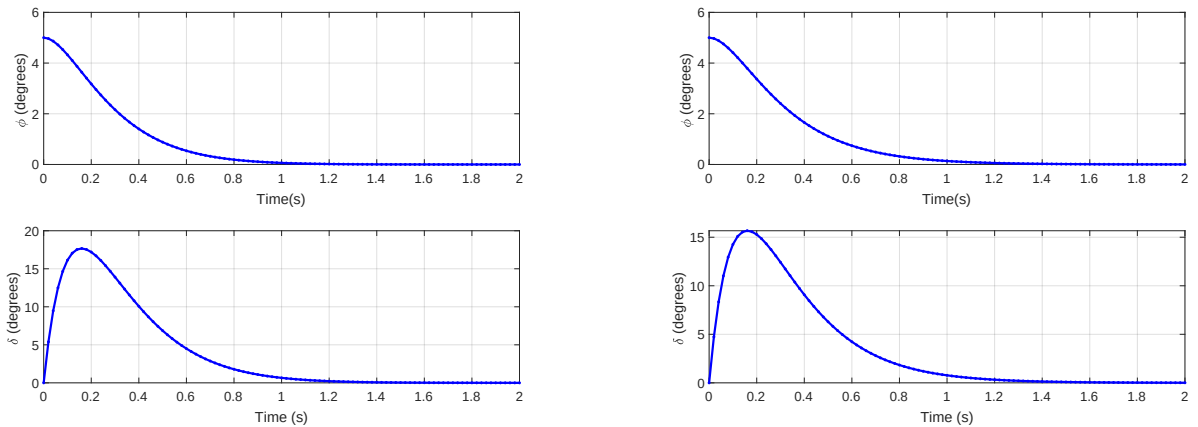
$$\omega_i \sim \mathcal{U} \left[-\frac{\epsilon}{\sqrt{n}}, \frac{\epsilon}{\sqrt{n}} \right]^n$$

to satisfy $\|\omega_i\|_\infty \leq \frac{\epsilon}{\sqrt{n}}$.

The data-driven state-feedback controller based on this data stabilised the bicycle efficiently and had similar or near coefficients to those of the discretised LQR.

Later, the output feedback gain vector from the LQR was applied on the real bicycle to collect some real data, instead of the synthesised data, for simulating and testing the data-driven state-feedback controllers. It turned out not to work since the process noise ϵ was too large for various segments of the data collected. From this, it was obvious that an online version of the data-driven state-feedback controllers would not work. This challenge is based on the real system errors compared to the synthesised data errors which were always bounded as needed.

In Figure 3.2, the simulation results for different fork angles under identical settings are presented. The feedback gains are comparable to those obtained from the LQR controllers, demonstrating that the controller is capable of balancing the bicycle.



(a) Data-driven state-feedback for 75° fork angle.

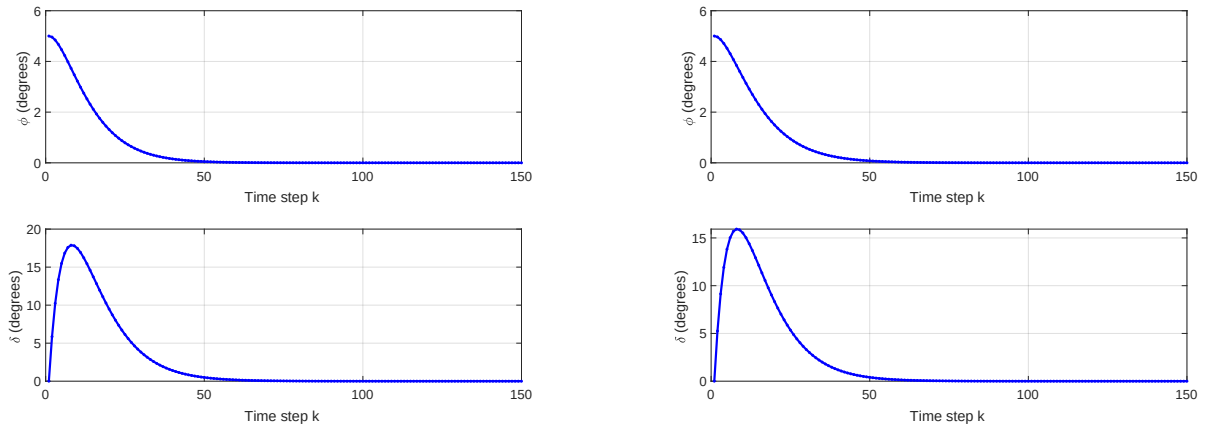
(b) Data-driven state-feedback for 90° fork angle.

Figure 3.2: Lean and steering angles over time under different data-driven state-feedback settings for an initial state vector (in radians) $x_0 = [0.0873, 0, 0]^\top$, $Q = \text{diag}(300, 0.1, 300)$, $R = 1$, noise bound $\epsilon = 0.001$ and data length of $T = 40$.

3.3 Data-Driven Min-Max MPC Controller

Finally, a full data-driven min-max MPC controller was implemented using synthesised input-state data similar to the previous controller. Here, the SDP (2.14) was solved in a receding horizon manner.

The simulation results for different fork angles under the same parameters are shown in Figure 3.3. The error bound ϵ is decreased to 10^{-5} to ensure obtaining a solution for the SDP at each iteration of the MPC.



(a) Data-driven MPC for 75° fork angle.

(b) Data-driven MPC for 90° fork angle.

Figure 3.3: Lean and steering angles over time under different data-driven MPC settings for an initial state vector (in radians) $x_0 = [0.0873, 0, 0]^T$, $Q = \text{diag}(300, 0.1, 300)$, $R = 1$, noise bound $\epsilon = 10^{-5}$, data length of $T = 40$, and $\text{mpc_iterations} = 150$.

3.4 Comparisons?

Add a Table for comparing costs here

Change this

The simplified model, despite neglecting the fork geometry, proved more effective for fast controller design and practical implementation on resource-constrained platforms. This unexpected behaviour is probably resulting from the unknown parameters of the bicycle and interaction with the environment.

4 Experimental Work

To test the data-driven controller in a real life experiment, a model-scale bicycle was chosen. The self-balancing mini-bike developed by **WHEELTEC** was utilized as the experimental platform. The primary goal of the project is to implement the data-driven controllers for stabilizing the bike. The mini-bike is equipped with various sensors and actuators to enable control and data acquisition.

Add Tapken's work on the same bike with simulink modelling for it. No: in intro better

4.1 Mini-Bike

The mini-bike is based on the **WHEELTEC Z100 Servo Self-Balancing Bicycle** in Figure 4.1. It features an STM32F103C8T6 microcontroller as the primary control unit, which integrates different sensors and motor control circuits to achieve stabilization.

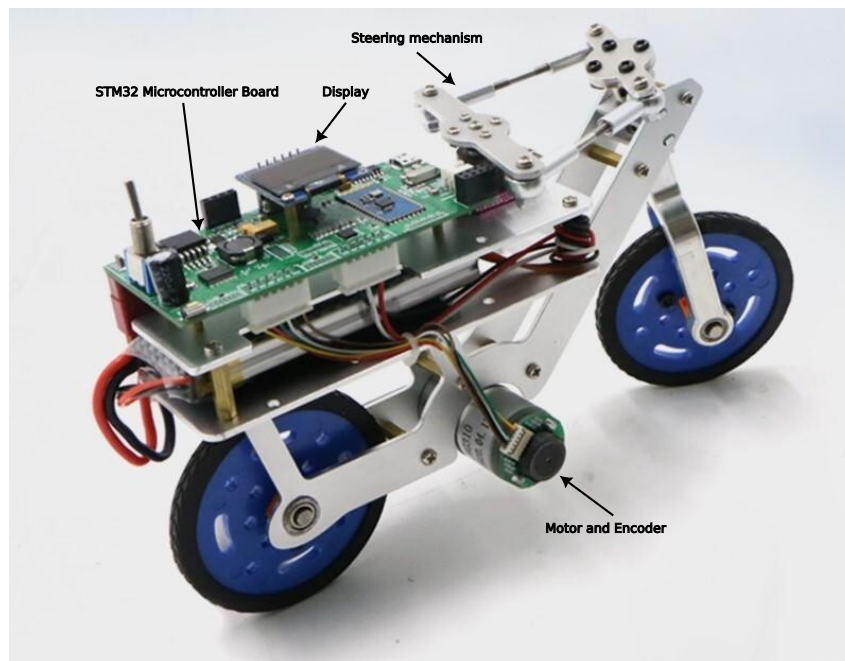


Figure 4.1: A picture for the MiniBike used in the experiments.

The bike has the following key components:

- **Microcontroller:** STM32F103C8T6 (ARM Cortex-M3)
- **IMU Sensor:** MPU6050 (3-axis gyroscope and accelerometer)
- **Servo Motor:** WHEELTEC Z100 steering servo
- **Rear Wheel Motor:** DC motor with incremental encoder

- **Motor Driver:** AT8236 H-bridge circuit
- **Power Supply:** 2S lithium-ion battery (7–12 V)
- **Wireless Communication:** Bluetooth module (USART3)

The bike uses steering angle control to self-balance. This is done through an LQR followed by a PID controller. The LQR computes the state-feedback gain that minimises the cost function. While the PID controller adjusts the PWM signals to the servo motor to fine-tune the steering speed.

The bike is balanced continuously through getting data from the gyroscope, which provides real-time data on tilt angles and angular velocities. The microcontroller operated at a frequency of 100 Hz. The measurements are processed by the microcontroller to compute the control law, where the input is the steering angle derivative.

The Data was collected from the bike through a Bluetooth connection using USART3, where the frequency of data was about 10 Hz. A mobile device was connected to the bike where the data received was saved for processing.

The mini-bike allows adjustment of the balance midpoint through a potentiometer; if it is offset from zero, the bike will move in a circle in the direction of the lean. Moving in a circle allowed for a longer range of data collected while driving under a controller.

The control algorithms were implemented on the STM32 using C++ programming with HAL Drivers. The platform on which the development of codes were done is the *stm32cube programmer*, which is a free unlimited software for writing and building the codes. It compiled the codes into a Hexadecimal file which can then be flashed on the STM32 microcontroller using FlyMCU software.

The codes read the IMU data (lean angle and angular velocity) via I2C communication. Then, the forward velocity is estimated by processing the encoder pulses. The lean angle is estimated by a complementary filter. The implemented control law is then applied using the states and a control input is determined. Consequently, the PWM signal of the steering servo is computed and applied.

The data-driven MPC controller was not implemented since the real-life errors prevented successful solving of the SDP on real data as stated before. Due to this limitation, the SDP optimisation was first performed offline, and the resulting state-feedback gain matrix was embedded into the embedded controller as a static gain.

The system states that were recorded are the lean angle (ϕ), lean rate ($\dot{\phi}$), and Steering angle (δ).

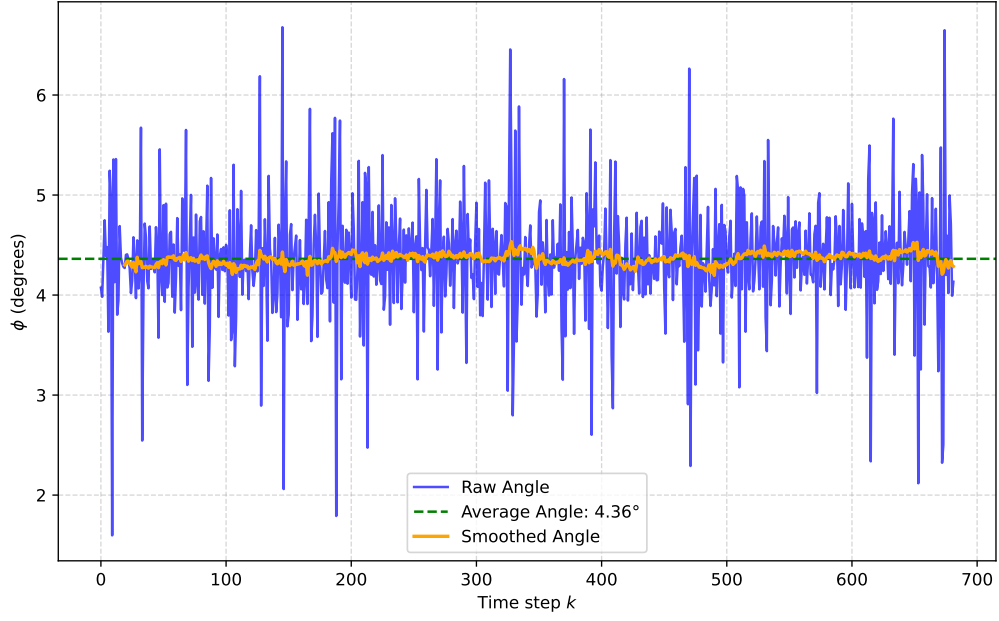
4.2 Experiments

Four different controllers were applied on the mini-bike based on both the LQR and data-driven state-feedback discussed in chapter 3. The experiments are described in Table 4.1.

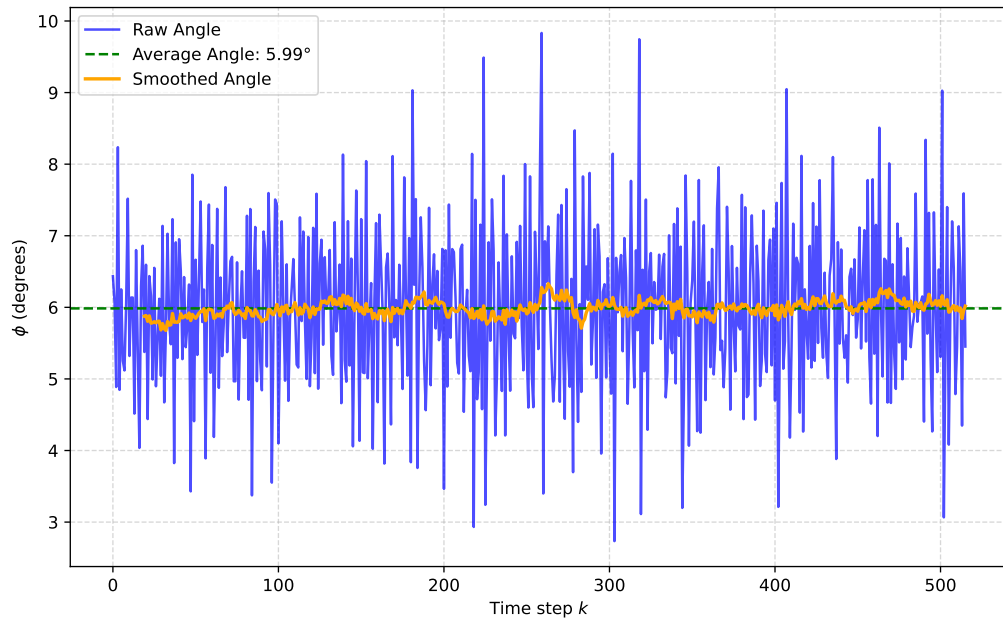
Table 4.1: Overview of controllers applied on the mini-bike

Experiment	Controller	Target Angle	Target Angle
1	LQR	75°	4°
2	LQR	90°	5°
3	Data-driven state-feedback	75°	3.5°
4	Data-driven state-feedback	90°	4°

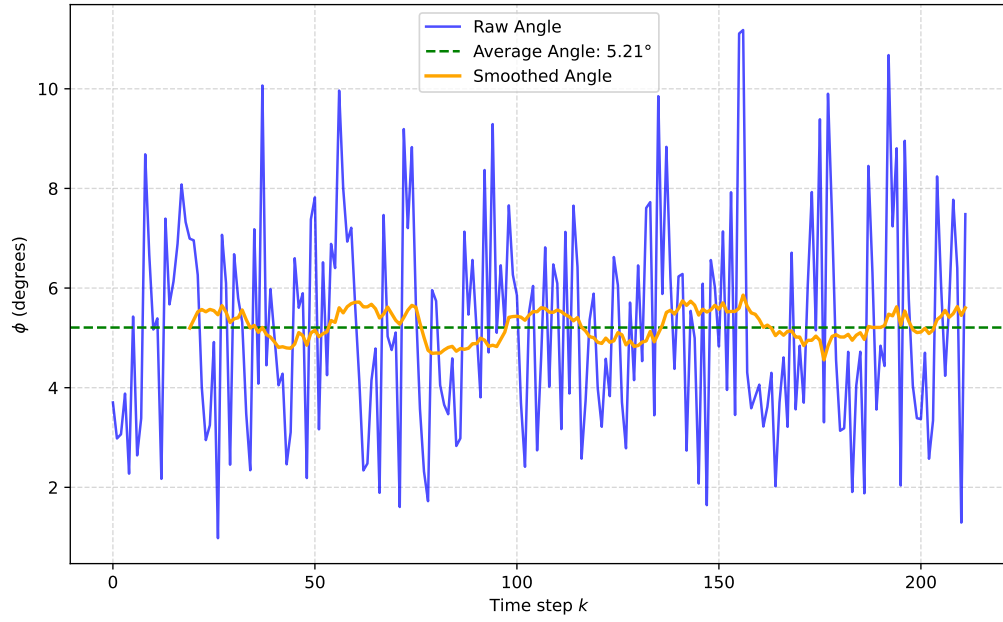
The results acquired on implementing the controllers and collecting the data are plotted in Figure 4.2. It can be noticed that the data length varies across the 4 experiments. This is due to the capability of collecting the data before the bicycle falls, particularly in experiment 3 in Figure 4.2c, where the controller did not maintain bicycle balance for an extended period.



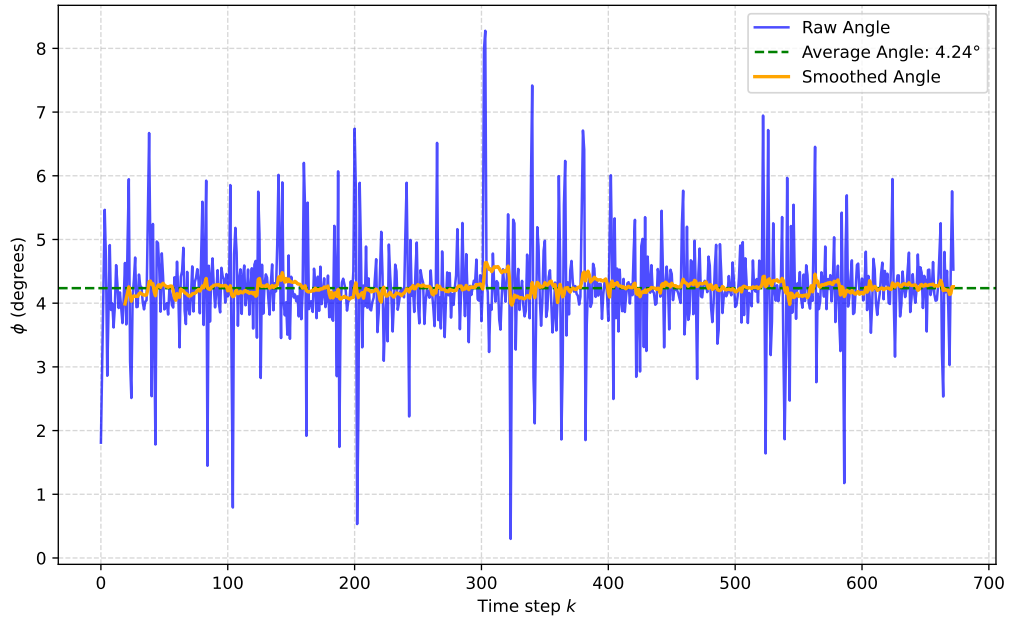
(a) LQR for 75° fork angle and 4° target angle.



(b) LQR for 90° fork angle and 5° target angle.



(c) Data-driven state-feedback for 75° fork angle and 3.5° target angle.



(d) Data-driven state-feedback for 75° fork angle and 4° target angle.

Figure 4.2: Lean angle over time under different data-driven state-feedback settings for an initial state vector (in radians) $x_0 = [0.0873, 0, 0]^\top$, $Q = \text{diag}(300, 0.1, 300)$, $R = 1$, noise bound $\epsilon = 0.001$ and data length of $T = 40$.

5 Conclusion and Future work

5.1 Conclusion

In this research project, the goal was to implement a data-driven MPC and test it for balancing a bicycle. Two models for the bicycle were studied which were simple second-order systems with and without considering the fork angle. The theoretical basis for the data-driven was studied and discussed where the data-driven min-max MPC was derived, narrowed down to state-feedback, then reformulated to be solved as an SDP in a sliding horizon manner. Following that, the data-driven controller was implemented in Matlab, modified for the models of the bicycle along with the input state constraints, and then simulated for two versions of the data-driven controller. The first version solved the SDP once, and then the controller was passed to the bicycle model in simulation. The second version solved the SDP as an MPC problem with shifting horizon. The real-life experiments were then carried on the WHEELTEC Mini-Bike by implementing 4 controllers. An LQR controller with two versions for both the fork and no-fork angle model of the bicycle. Additionally, a data-driven state-feedback based on a one-shot solution of the SDP was applied on the bicycle with its two models versions.

5.2 Future Work

For - Decrease errors (epsilon) (collect better data)

- Consider LPV system, with variable velocity!

- Implement online DDMPC - Add SDP robustness Potential improvements and future research directions include:

- Online adaptive Data-Driven MPC to incorporate real-time data updates.
- Implementation of rear-wheel steering stabilization for stationary balancing.
- Integration of higher precision IMUs to reduce noise.
- Hardware upgrades to allow full real-time online optimization on embedded platforms.
- Decrease

6 Conclusion

In this work, a Data-Driven Min-Max Model Predictive Control (DD-MPC) strategy was designed, implemented, and experimentally validated for the stabilization of a self-balancing mini-bicycle. Unlike traditional model-based methods, the data-driven approach leveraged collected input-state measurements directly, eliminating the need for an explicit mathematical model.

Through simulations and real-world experiments, the proposed Data-Driven MPC controller demonstrated improved performance compared to a baseline Linear Quadratic Regulator (LQR) controller. Specifically, the data-driven controller achieved faster stabilization, lower overshoot, and better robustness against model uncertainties and external disturbances.

Despite hardware limitations, such as computational restrictions on the STM32 microcontroller and sensor noise, the controller effectively stabilized the mini-bicycle within defined safe operating limits. The offline optimization approach combined with precomputed state-feedback gains enabled real-time embedded operation, confirming the feasibility of deploying advanced data-driven control strategies on resource-constrained platforms.

Future Work

Future research directions include:

- Development of an online adaptive Data-Driven MPC to update the controller in real-time based on newly collected data.
- Exploration of rear-wheel steering strategies for stationary balancing without forward motion.
- Integration of higher-performance embedded hardware to allow fully online SDP optimization onboard.
- Testing the controller under varying payload conditions and dynamic environments to further evaluate robustness.

Overall, the results of this research highlight the significant potential of data-driven control methods for autonomous vehicle stabilization, particularly in cases where traditional system identification is challenging or impractical.

Notes

■ Should be moved to Control?	2
■ This down here is mainly a control paragraph followed by modelling again .	2
■ This down here is mainly a repeated intro, modify it and remove what is redundant	2
■ Following paragraphs not convincing	2
■ may need fixes	3
■ acronyms	3
■ Check if data-driven	5
■ Add the Fork Model simple drawing	8
■ Check acronym	8
■ mention quadratic matrix inequality (QMI)	10
■ I am here now	15
■ Add a Table for comparing costs here	17
■ Change this	17
■ Add Tapken's work on the same bike with simulink modelling for it. No: in intro better	18

Acknowledgment

First and foremost, I would like to express my sincere gratitude to my supervisor M.Sc. Yifan Xie. I am deeply thankful for her insightful discussions, theoretical advice, and support with ideas during the whole research project period. Her mentorship and dedication were invaluable in overcoming the practical challenges encountered.

I would also like to extend my thanks to my examiners, Prof. Dr.-Ing. Andrea Iannelli and Jun.-Prof. Dr.-Ing. Florian Pfaff, for their feedback and ideas at the beginning and mid the project.

I would like to thank the Institute of System Theory and Automatic Control (IST) at the University of Stuttgart for hosting me and providing the lab facilities and the stimulating academic environment.

Finally, I am profoundly grateful to my family and friends for their unwavering encouragement, understanding, and support during my studies.

Bibliography

- [1] Masahiro Kawaguchi and Masaki Yamakita. "Stabilizing of bike robot with variable configured balancer". In: *SICE Annual Conference 2011*. 2011, pp. 1057–1062.
- [2] Jiarui He and Mingguo Zhao. "Control System Design of Self-balanced Bicycles by Control Moment Gyroscope". In: *Proceedings of the 2015 Chinese Intelligent Automation Conference*. Ed. by Zhidong Deng and Hongbo Li. Berlin, Heidelberg: Springer Berlin Heidelberg, 2015, pp. 205–214. ISBN: 978-3-662-46466-3.
- [3] F. J. W. Whipple. "The stability of the motion of a bicycle". In: *Quarterly Journal of Pure and Applied Mathematics* 30 (1899), pp. 312–348.
- [4] D.J.N. Limebeer and R.S. Sharp. "Bicycles, motorcycles, and models". In: *IEEE Control Systems Magazine* 26.5 (2006), pp. 34–61. DOI: 10.1109/MCS.2006.1700044.
- [5] Tom Andersson et al. "A Loop Shaping Method for Stabilising a Riderless Bicycle". In: *2019 European Conference on Mobile Robots (ECMR)*. 2019, pp. 1–6. DOI: 10.1109/ECMR.2019.8870965.
- [6] Chih-Keng Chen and Trung-Kien Dao. "A study of bicycle dynamics via system identification". In: *2010 International Symposium on Computer, Communication, Control and Automation (3CA)*. Vol. 2. 2010, pp. 204–207. DOI: 10.1109/3CA.2010.5533583.
- [7] M. A. Anjumol and V. R. Jisha. "Optimal stabilization and straight line tracking of an electric bicycle". In: *2014 International Conference on Power Signals Control and Computations (EPSCICON)*. 2014, pp. 1–6. DOI: 10.1109/EPSCICON.2014.6887487.
- [8] Vito Cerone, Davide Andreo, and Mats Larsson. "Stabilization of a Riderless Bicycle A Linear-Parameter-Varying Approach". In: URL: <https://api.semanticscholar.org/CorpusID:14769320>.
- [9] C. Tapken, A. Gloesekoetter, and J. Gloesekoetter. "Modelling and Controlling the Kinetic and Dynamic of a Bicycle". In: *Enseñanza y Aprendizaje de Ingeniería de Computadores* 8 (2018), pp. 97–113. URL: https://digibug.ugr.es/bitstream/10481/53302/1/T8_N8_Revista_EAIC_2018.pdf.
- [10] Niklas Persson et al. "A Comparative Analysis and Design of Controllers for Autonomous Bicycles". In: *Proceedings of the 2021 European Control Conference (ECC)*. IEEE, 2021, pp. 1570–1576. DOI: 10.23919/ECC54610.2021.9655083. URL: https://www.es.mdu.se/pdf_publications/6175.pdf.

- [11] Alen Turnwald and Steven Liu. “Motion Planning and Experimental Validation for an Autonomous Bicycle”. In: *IECON 2019 - 45th Annual Conference of the IEEE Industrial Electronics Society*. Vol. 1. 2019, pp. 3287–3292. DOI: 10.1109/IECON.2019.8927829.
- [12] Yongchao Yu and Mingguo Zhao. “Steering Control for Autonomously Balancing Bicycle at Low Speed”. In: *2018 IEEE International Conference on Robotics and Biomimetics (ROBIO)*. 2018, pp. 33–38. DOI: 10.1109/ROBIO.2018.8665347.
- [13] Yifan Xie, Julian Berberich, and Frank Allgöwer. “Data-Driven Min-Max MPC for Linear Systems”. In: *2024 American Control Conference (ACC)*. 2024, pp. 3184–3189. DOI: 10.23919/ACC60939.2024.10644295.